

A Good Rubber Duck Does Not Quack

Designing Socratic Scaffolding in AI Tutors

Ayush Thonge

ayush.thonge_ug2025@ashoka.edu.in

Ashoka University

Sonipat, Haryana, India

Aalok Thakkar

thakkar@ashoka.edu.in

Ashoka University

Sonipat, Haryana, India

Abstract

Large Language Models in programming education face a fundamental tension: while capable of generating correct solutions instantly, doing so undermines the struggle necessary for conceptual learning. We present Socratic AI, a VS Code-integrated tutor that addresses this through pedagogically-grounded Socratic dialogue constrained to withhold direct solutions.

Our core innovation is a stateful misconception detection mechanism that tracks student understanding across dialogue turns. Operating on a taxonomy of foundational programming errors (off-by-one, mutation vs. reassignment, return vs. print, async/parallel confusion), the system evaluates each student utterance to produce per-misconception verdicts (reinforced, weakened, introduced, absent) with certainty scores. These accumulate through small, incremental confidence score updates until a derived resolution threshold is achieved. This enables the tutor to persist on specific conceptual gaps rather than abandoning them when students deflect or express frustration. This stateful tracking enables two key capabilities: (i) the system adapts questioning intensity based on detected learner confidence signals, shifting from diagnostic probes to conceptual contrasts, and (ii) enforces purely Socratic constraints through multi-layered validation that blocks code generation, direct explanations, and multi-question responses. The system is implemented as a VS Code extension with Gemini-based classification back-end and cloud database persistence for longitudinal analysis.

In a preliminary deployment with 48 students, we observed increased question specificity, frequent learner articulation consistent with mental model revision, and reduced recurrence of targeted errors on similar tasks. We contribute a deployable architecture, a misconception taxonomy, and evidence that stateful tracking enables adaptive Socratic dialogue that scaffolds productive struggle rather than short-circuiting learning.

CCS Concepts

• **Applied computing** → **Interactive learning environments**; • **Social and professional topics** → *Computing education*.

Keywords

AI tutoring systems, Programming education, Adaptive scaffolding, Socratic dialogue

ACM Reference Format:

Ayush Thonge and Aalok Thakkar. 2026. A Good Rubber Duck Does Not Quack: Designing Socratic Scaffolding in AI Tutors. In *Proceedings of the 31st ACM Conference on Innovation and Technology in Computer Science Education V. 1 (ITiCSE 2026)*, July 10–15, 2026, Madrid, Spain. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3803400.3809368>

1 Introduction

Programming education frequently produces learners who can reproduce familiar code patterns but struggle to construct solutions or debug novel failures. This phenomenon, often described as procedural fragility, reflects a mismatch between surface fluency and underlying conceptual understanding. Students may complete assignments successfully while lacking robust mental models of control flow, state, or abstraction.

Learning to program requires productive struggle: iterative cycles of prediction, execution, observation, and revision. Through these cycles, learners test hypotheses about program behaviour, confront mismatches between expectation and outcome, and refine their understanding. However, productive struggle is cognitively demanding and time-intensive. As a result, learners often seek shortcuts: historically through peers or online forums, and increasingly through AI coding assistants.

Contemporary LLM-based tools dramatically lower the cost of bypassing struggle by providing fluent, complete solutions on demand. While effective for professional productivity, answer-first assistance risks undermining learning by replacing generative reasoning with selection and recognition. Learners may accept plausible code without understanding why it works, reinforcing illusions of competence and encouraging shallow debugging strategies.

A further challenge is that novice programming errors are systematic rather than idiosyncratic. Decades of computing education research document recurring misconceptions, including off-by-one errors, confusion between mutation and reassignment, misuse of return values, incorrect assumptions about execution order, and scope-related errors. Correcting such misconceptions requires targeted conceptual intervention, not merely exposure to correct code.

This paper explores an alternative design point for AI assistance in programming education: an IDE-embedded tutor constrained to Socratic questioning. Rather than optimizing for correctness or speed, the system is intentionally limited to preserve learner reasoning. Central to this design is a stateful representation of learner misconceptions that persists across dialogue turns, enabling adaptive and focused questioning.

We address the question: Do interactions with a Socratic AI tutor correlate with reduced recurrence of common programming errors?



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

ITiCSE 2026, Madrid, Spain

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2634-7/2026/07

<https://doi.org/10.1145/3803400.3809368>

2 Background and Related Work

This work draws on several strands of computing education research: theories of productive struggle and self-explanation, Socratic tutoring and Intelligent Tutoring Systems (ITS), and recent work on AI-powered coding assistants in educational contexts. We situate Socratic AI at the intersection of these traditions.

Productive struggle is widely recognized as a critical mechanism for durable learning in complex cognitive domains. Rather than minimizing difficulty, productive struggle involves learners actively grappling with generating hypotheses, testing predictions, and revising mental models. In programming, such struggle supports the construction of causal models of execution, including control flow, state change, and data abstraction [3, 4]. Research in CS education has consistently shown that students who engage in prediction, tracing, and explanation develop deeper understanding than those who focus solely on obtaining correct outputs [12, 13]. Work on self-explanation demonstrates that prompting learners to articulate their reasoning, even when incomplete or incorrect, improves transfer and retention by making implicit assumptions explicit and exposing contradictions that require resolution. However, productive struggle is fragile. When external help is too directive or too complete, learners bypass the generative processes. This tension is especially salient in programming, where a single correct solution can be copied and run without understanding. Our work treats struggle not as an obstacle to be removed, but as a resource to be scaffolded.

Socratic tutoring emphasizes learning through sequenced questioning that guides learners to articulate reasoning, confront inconsistencies, and refine conceptual models. Rather than transmitting answers, Socratic tutors focus on eliciting the learner’s own explanations and predictions. In programming education, this approach has been used to surface misconceptions about variables, control flow, and program state that often remain hidden when feedback is limited to correctness [2, 12].

Several prior systems have explored Socratic or dialogue-based tutoring, though in limited ways. Alshaikh and Cakmak’s Socratic tutor for source code comprehension showed great promise for the question-driven interaction that could improve student explanation quality in structured lab settings, but operated on fixed question templates without persistent learner state [1]. Harvard’s CS50 Duck Debugger applies a conversational debugging style that encourages students to narrate their code out aloud, but responses are stateless [10]. Each turn is independent causing memory leakage and the system does not track which conceptual errors persist across a student’s sessions. Other similar dialogue based ITS such as Auto-Tutor uses persistent learner models but require extensive manual authoring of dialogue scripts and are not specifically designed for programming tasks [7].

Socratic AI differs from these approaches in two ways: it combines the versatility of LLMs to tackle questions with a persistent, probabilistic misconception model that accumulates evidence across turns [14]. This allows it to first address the core misconception that the student is facing and second, generate unique questions on demand without requiring pre-specified dialogue trees.

ITS operationalize these ideas through explicit representations of domain knowledge, learner state, and pedagogical policy. Despite

their pedagogical effectiveness, traditional ITS face well-documented limitations [2, 9]. They typically require extensive manual authoring of domain models, hint sequences, and problem representations, making them expensive to build and difficult to extend beyond narrowly scoped curricula.

More recent systems have explored conversational or dialogue-based tutoring, including debugging assistants that prompt learners to reflect on program behaviour. Notably, Harvard’s CS50 Duck Debugger adopts a conversational, question-oriented style inspired by rubber duck debugging, encouraging students to explain their code line by line [10]. While such systems move toward Socratic interaction, they generally lack persistent learner models and do not explicitly track misconceptions across turns. In general, these LLM-based coding assistants excel at generating fluent explanations and complete solutions. In educational settings, they have been shown to improve short-term task completion and reduce frustration [6], however, a growing body of computing education research raises concerns about their pedagogical impact. Answer-first assistance can reduce opportunities for generative practice, encourage shallow trial-and-error debugging, and foster overreliance [5, 11]. Learners may mistake recognition of a plausible solution for understanding, leading to illusions of competence. Despite several recent studies and calls for AI tools that better align with instructional goals, most LLM-based assistants remain fundamentally optimized for solution delivery rather than for sustaining productive struggle [5].

Our work bridges Socratic tutoring and LLM-based systems by combining the flexibility and coverage of language models with explicit pedagogical constraints and lightweight learner modelling. Unlike answer-first coding assistants, Socratic AI is designed in a way to provide a question-only approach and restrained from providing answers under any circumstance. The question generated relies heavily on the type of misconception that the student is facing with the end goal of eventually leading them to an answer. By embedding Socratic scaffolding directly into programming workflows, this work contributes to an emerging research agenda on how AI systems can support, rather than short-circuit, the development of programming reasoning skills.

3 Design Goals and Pedagogical Rationale

The design of Socratic AI is guided by learning-science principles from CS education literature [3, 4, 8]. Rather than treating AI assistance as a neutral productivity aid, we explicitly encode pedagogical values into the system’s affordances and constraints.

3.1 Preserve Generative Reasoning

Learning to program requires learners to design solutions, not merely recognize correct ones. This involves formulating hypotheses about program behaviour, tracing execution paths, anticipating state changes, and selecting among plausible implementations. When a system provides complete solutions or explicit fixes, these cognitive activities are bypassed, shifting learners from generation to selection. To counter this effect, Socratic AI prohibits code generation, explicit fixes, and step-by-step explanations. The system does not “complete” the learner’s task; instead, it maintains the

learner as the primary agent responsible for proposing, testing, and revising ideas.

3.2 Promote Articulation and Reflection

Articulation serves as a step where learners explain expected program behaviour, state transitions, or causes of failure. They externalize tacit assumptions that would otherwise remain unexamined. Socratic AI is designed to repeatedly elicit articulation at moments of uncertainty. Questions are framed to prompt learners to predict outcomes (“What value do you expect this variable to hold after the loop?”) or to justify interpretations (“Why do you think this function returns None here?”). The emphasis on articulation supports metacognitive awareness: by requiring learners to put reasoning into words, the system encourages them to notice gaps, contradictions, or ungrounded assumptions in their own thinking. This helps shift behaviour from reactive trial-and-error toward more deliberate hypothesis-driven exploration.

3.3 Support Situated Learning in the IDE

Socratic AI is embedded within the IDE to align with learners’ natural workflow. The system can reference the learner’s actual variables, control structures, and execution paths, grounding questions in the immediate context of work. This integration enables short, lightweight interactions that mirror how programmers naturally reflect while debugging.

3.4 Enable Persistent Conceptual Focus

Novice programming misconceptions are often resilient. Learners may temporarily produce correct code while retaining flawed mental models, leading to recurring errors in new contexts. Socratic AI is designed to persist on unresolved conceptual gaps across dialogue turns. Rather than treating each question in isolation, the system maintains a stateful representation of which misconceptions are likely active. This allows the tutor to continue probing a specific concept until there is evidence that the learner’s understanding has meaningfully changed.

Persistence does not imply rigidity. As confidence in a misconception weakens, the system adapts the intensity and type of questioning. However, the system resists abandoning a conceptual thread solely due to deflection, impatience, or superficial success, reflecting the view that conceptual repair requires sustained attention.

4 System Architecture

Socratic AI is implemented as a distributed tutoring system that deliberately separates learner interaction from pedagogical reasoning. This separation is a pedagogical design choice intended to ensure that instructional behaviour remains stable, interpretable, and aligned with Socratic principles, even when powered by a stochastic large language model.

The system consists of two primary components: (1) a learner-facing IDE extension and (2) a cloud-based pedagogical service that maintains learner state and governs all instructional decisions.

4.1 Learner-Side IDE Integration

The learner-facing component is implemented as a Visual Studio Code extension. It manages all user interaction, including tutor

invocation, display of questions, and lightweight dialogue continuity. The extension captures only limited, permissioned context—typically a selected code region or a brief reference to recent execution output—to ground questions in the learner’s immediate task without performing static analysis or automated debugging.

Crucially, the IDE client performs no pedagogical reasoning. It does not classify learner responses, select hints, or generate questions. All such decisions are deferred to the back-end service. This design prevents instructional logic from being entangled with interface concerns and allows pedagogical behaviour to remain invariant across IDE sessions, languages, and model updates.

4.2 Pedagogical Service and Control Flow

The cloud-based pedagogical service mediates all interaction with the underlying language model. It is responsible for (i) analysing learner utterances for evidence of conceptual understanding or misunderstanding, (ii) maintaining a persistent representation of learner misconceptions across turns, (iii) selecting pedagogical moves, and (iv) enforcing strict Socratic constraints on all generated responses.

Treating the language model as an untrusted generator rather than an autonomous tutor is a deliberate choice. All model outputs are post-processed and validated before being returned to the learner. This allows the system to remain consistent even as the underlying model changes or evolves.

4.3 Interaction Loop

Each tutoring turn follows a fixed interaction loop designed to preserve learner agency and promote generative reasoning:

- (1) **Learner initiation.** The learner invokes the tutor from within the IDE, optionally providing a selected code fragment or describing a recent program behaviour.
- (2) **Misconception state update.** The learner’s utterance is analyzed for evidence related to known conceptual misconceptions, updating a persistent learner state.
- (3) **Pedagogical move selection.** Based on the updated state, the system selects a single pedagogical move, typically a focused Socratic question targeting the most salient unresolved misconception.
- (4) **Learner response.** The learner responds by articulating reasoning, revising an explanation, or testing a hypothesis through code modification.

This loop repeats until the system detects sufficient evidence that a targeted misconception has weakened or until a stopping condition is reached. Rather than terminating with a solution, the system concludes with a brief confirmation or reflective prompt, reinforcing learner ownership of the resolution (see Tables 2 and 3 for illustrative examples).

4.4 Misconception Taxonomy and State Model

At the core of Socratic AI is an explicit taxonomy of foundational programming misconceptions. The taxonomy is derived from prior computing education research and instructional practice and is intentionally coarse-grained. Rather than modelling problem-specific skills, it targets conceptual misunderstandings that recur across

languages, paradigms, and tasks. Table 1 summarizes the misconception classes supported in the current implementation.

Each misconception is represented internally as a continuous confidence value in the interval $[0, 1]$, reflecting the system’s estimate that the misconception is currently active for the learner. These values are not interpreted as strict probabilities, but as pedagogically meaningful indicators of conceptual risk.

After each learner utterance, the system produces a per-misconception verdict: *reinforced*, *weakened*, *introduced*, or *absent*, based on structured classification of the learner’s explanation and referenced code context. Confidence updates are bounded and incremental, ensuring stability under noisy or ambiguous learner responses. This allows weak signals to accumulate across turns and prevents the system from abandoning a conceptual focus after superficial correctness or learner deflection.

This stateful representation serves as a lightweight learner model, enabling persistent targeting without requiring the extensive domain modelling typical of traditional intelligent tutoring systems.

4.5 Constraint Enforcement and Failure Modes

To ensure adherence to Socratic pedagogical principles, all model-generated outputs are subjected to multiple layers of constraint enforcement. Responses are validated to prohibit:

- generation of code snippets or concrete implementations,
- explicit fixes or corrective instructions,
- step-by-step procedural explanations,
- multi-question or compound prompts.

Outputs that violate these constraints are rejected or regenerated before being shown to the learner. Each valid response is limited to a single focused question or a brief acknowledgment, preventing explanation cascades that could supplant learner reasoning.

These constraints necessarily introduce trade-offs. In time sensitive debugging scenarios, single-question Socratic prompts may feel slower or less immediately helpful than direct explanations. Additionally, misconception classification relies on probabilistic judgments and may occasionally misattribute learner intent, leading to suboptimal question targeting. Rather than eliminating these risks, the system is designed to mitigate them through bounded confidence updates and persistence across turns, favouring conceptual stability over reactivity.

Together, these architectural choices operationalize a central design claim of this work: that even a lightweight, misconception-driven learner state when combined with pedagogical constraints can alter the behaviour of tutors, shifting them from solution engines toward tools that scaffold durable understanding.

4.6 Misconception Confidence Update Rules

The update rules are asymmetric by design. A reinforced verdict adds 0.22 to a misconception’s confidence, while a weakened verdict subtracts only 0.18. This reflects the pedagogical principle that a single correct response is insufficient evidence of durable understanding. It is easier to introduce a misconception than to resolve one. Misconceptions begin at a neutral confidence of 0.32. When first detected, an additional 0.11 is added, placing newly introduced misconceptions at 0.43. When a misconception goes unmentioned in a turn, its confidence decays passively by 10%, preventing stale

misconceptions from persisting when a student moves to a different topic. A misconception is considered resolved and removed from the learner’s state once its confidence falls below 0.18. All values are clamped to the interval $[0, 1]$.

5 Usage Context and Observations

Socratic AI was deployed in a limited, exploratory setting. The system was used in two sections of an introductory programming course, involving a total of 48 undergraduate students at Ashoka University. Students engaged with Socratic AI during regular programming activities, primarily while debugging or refining solutions to short programming exercises. Use of the system was optional and supplementary; it did not replace instructor feedback, peer discussion, or existing instructional materials.

The deployment spanned a full semester, allowing individual students to interact with the tutor across multiple sessions and programming tasks. This exposure was critical for observing whether misconception-driven state tracking could persist meaningfully across turns and sessions.

5.1 Data Collection

Data collected during the deployment included anonymized dialogue transcripts, selected code context provided by students, and system-generated misconception state trajectories. No automated assessment scores or graded outcomes were used as primary measures. Instead, the focus was on interactional properties of the tool and on qualitative indicators of learner reasoning.

All observations reported here are intended to inform system design and future controlled studies rather than to support claims about learning effectiveness.

5.2 Observed Interaction Patterns

Several consistent interaction patterns emerged across students and sessions.

First, stateful misconception tracking enabled the tutor to maintain conceptual focus across turns, even when learners attempted to redirect the interaction toward solution-oriented help. For example, when learners asked for confirmation that a change “fixed the problem,” the tutor frequently responded by returning to an unresolved misconception (e.g., loop bounds or return semantics), prompting learners to articulate why the change worked rather than accepting surface correctness. This persistence contrasts with Socratic prompting that lacks memory of previous turns, which often abandons conceptual threads once a plausible answer appears.

Second, learners frequently shifted from terse, outcome-oriented queries (e.g., “Why is this not working?”) to more explanatory responses over the course of an interaction. In many sessions, students began to volunteer predictions about program behaviour or to justify design choices without being explicitly prompted to do so. While these behaviours do not constitute evidence of learning gains, they suggest that constraint-enforced Socratic questioning can reshape the discourse norms of AI-assisted debugging.

Third, the single-question constraint appeared to slow interaction tempo but increased question specificity. Rather than broad prompts or multi-part explanations, the tutor consistently asked

Table 1: Foundational programming misconception taxonomy

Misconception Class	Description
Off-by-one reasoning	Incorrect assumptions about loop bounds, indexing conventions, or termination conditions, including confusion between inclusive and exclusive ranges.
Mutation vs. reassignment	Failure to distinguish between modifying an object’s internal state and rebinding a variable name to a new value.
Return vs. print semantics	Misunderstanding the difference between producing a value for further computation and displaying output as a side effect.
Control flow misconceptions	Incorrect mental models of conditionals, early returns, or loop execution (e.g., assuming all branches execute).
Asynchronous vs. parallel execution	Erroneous assumptions about execution order, blocking behaviour, or concurrency in asynchronous or event-driven code.

Table 2: Example of a Socratic conversation clarifying scope and variable shadowing

Code Under Discussion:	
<pre>def change(x): x = 10 a = 5 change(a) print(a)</pre>	
Learner:	Why does a stay 5 even though x becomes 10 inside the function?
Tutor:	What is the difference between x and a?
Learner:	I think x is just a copy inside the function, and a is the original variable.
Tutor:	What happens to the copy of a after the function finishes executing?
Learner:	It is destroyed when the function ends, so a stays 5.

narrowly targeted questions tied to a single conceptual issue. Instructors observing the deployment noted that these questions resembled those posed during one-on-one human tutoring, particularly in their focus on justification rather than correction.

6 Discussion

This paper set out to explore whether LLMs can be repurposed from answer-first assistants into pedagogically aligned tutors by enforcing Socratic constraints and maintaining a lightweight, stateful learner model. We revisit our central research question:

Do interactions with a Socratic AI tutor correlate with reduced recurrence of common programming errors?

In a limited and exploratory sense, the answer appears to be affirmative: interactions with the system were associated with reduced recurrence of certain classes of programming errors within the scope of observed tasks. System logs and instructor observations suggest that once a misconception’s confidence decayed below resolution thresholds, learners were less likely to reintroduce the same class of error in subsequent interactions on similar problems. This

Table 3: Example of a Socratic conversation clarifying Mutation vs Reassignment

Code Under Discussion:	
<pre>def add_item(lst): lst = lst + [1] items = [] add_item (items) print (items)</pre>	
Learner:	Why is items still empty even though I added 1 inside the function?
Tutor:	What happens to lst after the function add_item finishes?
Learner:	It stops existing after the function ends, and items stays the same.
Tutor:	What happens to the new lst inside the function?
Learner:	It gets created inside the function and then gets thrown away when the function finishes.
Tutor:	How could the function modify the existing list instead of creating a new one?
Learner:	By mutating the passed list directly, or by returning the new list and assigning it back.

pattern is consistent with the hypothesis that sustained conceptual focus can reduce the likelihood of recurring novice errors.

However, this result should be interpreted cautiously. Our study design lacks a control condition; the observed reduction in error recurrence cannot be causally attributed to the Socratic intervention and may reflect natural learning progression, concurrent course instruction, or selection bias among voluntary users. The findings are therefore correlational and task-local, and do not establish claims about learning transfer or long-term retention.

Several features of the system appear to contribute to this pattern. First, the enforcement of Socratic constraints materially altered the character of interactions. Rather than providing broad explanations or multi-step fixes, the tutor consistently produced narrowly targeted questions focused on prediction, justification, or interpretation of program behaviour. This specificity was not achieved through prompting alone, but depended on architectural

enforcement mechanisms that constrained each response to a single pedagogical move and rejected outputs that violated Socratic structure. These interaction patterns align with prior concerns in CS education literature about over-explanation and premature resolution in answer-first AI systems.

Second, the use of a persistent, misconception-driven learner state appeared to support more sustained engagement with underlying concepts. The system’s ability to track and revisit unresolved misconceptions across turns encouraged learners to articulate reasoning more explicitly over time. In many sessions, learners shifted from outcome-oriented questions (e.g., whether a change “fixed” the program) toward explanations of why a construct behaved as it did. Rather than accepting superficial correctness, the tutor repeatedly returned to the same conceptual issue until learners articulated a coherent explanation or prediction. While we do not directly measure mental model change, these interactional shifts are consistent with prior research on self-explanation and productive struggle.

A key contribution of this work is therefore methodological: it identifies misconception persistence and decay as promising intermediate signals for evaluating AI-supported learning, which can be tested more rigorously in future controlled studies.

6.1 Design Implications

Taken together, these findings support the central design claim of this paper: that even a lightweight, stateful learner model—when coupled with strict Socratic constraints—can fundamentally change the behaviour of LLM-based tutors. The system resists several well-documented failure modes of answer-first assistants, including over-explanation, abandonment of unresolved conceptual gaps, and premature convergence on correct-but-unexplained solutions.

At the same time, the deployment surfaced important tensions. Some learners expressed frustration when the system declined to provide direct fixes, particularly under time pressure. Others attempted to minimize effort by providing shallow or evasive explanations, highlighting the limits of current articulation detection. These tensions reinforce the view that Socratic AI should be positioned as a complementary learning tool, selectively deployed in contexts where conceptual understanding is prioritized over immediate task completion.

6.2 Limitations

This work has several limitations that constrain the interpretation of its findings. First, the enforcement of single-question Socratic responses reduces flexibility in complex or time-sensitive debugging scenarios and may not be appropriate for all instructional contexts. There remains an inherent tension between maintaining Socratic rigour and supporting students under time pressure, which may lead to frustration or disengagement.

Second, the misconception taxonomy is coarse-grained and partially language-dependent, which can lead to misclassification or unstable targeting as learners shift topics or tasks. The taxonomy is also incomplete and hand-authored for introductory computer science. Potential paths forward include semi-automated taxonomy induction from student error logs and LLM-assisted expansion, though these were not explored here.

Third, misconception detection relies on probabilistic LLM judgments, and the heuristic confidence update rules are not calibrated against validated learning outcomes. As a result, resolution signals should be interpreted as pedagogical heuristics rather than definitive indicators. Fourth, our study design lacks a control condition; the observed reduction in error recurrence cannot be causally attributed to the Socratic intervention and may reflect natural learning progression or concurrent course instruction. More broadly, the study design does not support causal claims about learning gains, transfer, or long-term retention.

Finally, reliance on external LLM services raises unresolved concerns around privacy, cost, robustness, and scalability. In particular, scaling to larger deployments or more diverse subject areas may introduce additional engineering and resource constraints that were outside the scope of this work.

7 Future Work

This work opens several directions for future research. First, controlled studies comparing Socratic AI with answer-first assistants are needed to isolate the effects of constraint enforcement and stateful misconception tracking on learning outcomes. Second, richer learner models could incorporate additional signals, such as code edits, execution traces, or temporal patterns of interaction, to improve misconception detection and reduce susceptibility to shallow articulation.

Third, the misconception taxonomy itself can be expanded and empirically validated across languages, paradigms, and curricular contexts. Fourth, future systems could explore adaptive relaxation of Socratic constraints, allowing limited explanatory support once learners demonstrate sustained articulation and conceptual stability. Finally, longitudinal studies are needed to assess whether reduced recurrence of misconceptions translates into durable understanding and improved transfer.

8 Conclusion

This paper presented Socratic AI, a constrained AI tutoring system for programming education that prioritizes generative reasoning over answer delivery. By enforcing Socratic-only interaction and maintaining a stateful model of learner misconceptions, the system sustains conceptual focus across dialogue turns rather than optimizing for short-term correctness.

The central contribution of this work is a design argument: that pedagogically aligned AI tutors require explicit learner state and constraint enforcement, not merely better prompts or explanations. Our exploratory deployment demonstrates the feasibility and promise of this approach, showing that even lightweight misconception tracking can reshape AI-assisted interactions toward articulation, persistence, and conceptual repair.

Rather than positioning Socratic AI as a finished tool, this paper frames it as a proposal for a new design space in AI-assisted programming education. We argue that the success of future educational AI systems should be judged not by the correctness of the answers they provide, but by their ability to scaffold durable understanding and productive struggle.

Acknowledgement. We gratefully acknowledge the support of the Mphasis Foundation for funding this research.

References

- [1] Zeyad Alshaikh, Lasagn Tamang, and Vasile Rus. 2020. A Socratic Tutor for Source Code Comprehension. In *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics) (Lecture Notes in Artificial Intelligence, Vol. 12164)*. 15–19. doi:10.1007/978-3-030-52240-7_3
- [2] John R. Anderson, Albert T. Corbett, Kenneth R. Koedinger, and Ray Pelletier. 1995. Cognitive Tutors: Lessons Learned. *The Journal of the Learning Sciences* 4, 2 (1995), 167–207. doi:10.1207/s15327809jlst0402_2
- [3] Michelene T. H. Chi, Miriam Bassok, Matthew W. Lewis, Peter Reimann, and Robert Glaser. 1989. Self-Explanations: How Students Study and Use Examples in Learning to Solve Problems. *Cognitive Science* 13, 2 (1989), 145–182. doi:10.1207/s15516709cog1302_1
- [4] Michelene T. H. Chi, Nicholas de Leeuw, Mei-Hung Chiu, and Christian La-Vancher. 1994. Eliciting Self-Explanations Improves Understanding. *Cognitive Science* 18, 3 (1994), 439–477. doi:10.1207/s15516709cog1803_3
- [5] Paul Denny, Viraj Kumar, and Nasser Giacaman. 2023. ChatGPT and the Future of CS Education: A Conversation. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education (SIGCSE 2023)*. 1064–1070. doi:10.1145/3545945.3569751
- [6] James Finnie-Ansley, Paul Denny, Brett A. Becker, and Andrew Luxton-Reilly. 2022. Student Perceptions of AI-Assisted Programming Tools: Copilot Outperforms. In *Proceedings of the 2022 ACM Conference on International Computing Education Research (ICER)*. doi:10.1145/3501385.3543971
- [7] A. C. Graesser, P. Chipman, B. C. Haynes, and A. Olney. 2005. AutoTutor: an intelligent tutoring system with mixed-initiative dialogue. *IEEE Trans. on Educ.* 48, 4 (Nov. 2005), 612–618. doi:10.1109/TE.2005.856149
- [8] Manu Kapur. 2008. Productive failure. *Cognition and Instruction* (2008).
- [9] Kenneth R. Koedinger, John R. Anderson, William H. Hadley, and Mary A. Mark. 1997. Intelligent Tutoring Goes to School in the Big City. *International Journal of Artificial Intelligence in Education* 8 (1997), 30–43.
- [10] Rongxin Liu and David J. Malan. 2023. Implementing the CS50 Duck with OpenAI’s APIs. In *Proceedings of the IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. Related to CS50 Duck Debugger tool; confirmed presentation.
- [11] James Prather, Brett A. Becker, Paul Denny, James L. Pearce, Arif Alhadlaq, and Sebastian Stumpf. 2023. The Robots Are Coming: Exploring the Implications of Generative AI for Teaching and Learning Programming. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education (ITiCSE)*. 247–253. doi:10.1145/3587102.3595874
- [12] Elliot Soloway and James C. Spohrer (Eds.). 1986. *Studying the Novice Programmer*. Lawrence Erlbaum Associates, Hillsdale, NJ.
- [13] James C. Spohrer and Elliot Soloway. 1986. Novice Mistakes: Are the Folk Wisdoms Correct?. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 624–627. doi:10.1145/22627.22451
- [14] Karthik Sunil and Aalok Thakkar. 2025. SocraticAI: Transforming LLMs into Guided CS Tutors Through Scaffolded Interaction. arXiv:2512.03501 [cs.CY] <https://arxiv.org/abs/2512.03501>